LANGUAGES AND AUTOMATA I

HOMEWORK #5 PARTIAL ANSWERS

restaurant.go

```
// It's a restaurant simulation with customers placing orders and cooks
// preparing meals. The waiter can hold 3 orders at a time, and customers
// will wait for 5 seconds before abandoning an order if the waiter is busy.
// Each of the 10 customers will eat 5 meals before leaving the restaurant.
// Oh, and the three cooks, Remy, Colette, and Linguini, actually will deliver
// the meals directly to the customers.

package main

import (
        "log"
        "math/rand"
        "sync"
        "sync/atomic"
        "time"
)

// A little utility that simulates performing a task for a random duration.
// For example, calling do(10, "Remy", "is cooking") will compute a random
// number of milliseconds between 5000 and 10000, log "Remy is cooking",
// and sleep the current goroutine for that much time.

func do(seconds int, action ...any) {
    log.Println(action...)
    randomMillis := 500 * seconds + rand.Intn(500 * seconds)
    time.Sleep(time.Duration(randomMillis) * time.Millisecond)
}

// An order for a meal is placed by a customer and is taken by a cook.
// When the meal is finished, the cook will send the finished meal through
// the reply channel. Each order has a unique id, safely incremented using
// an atomic counter. It turns out the atomic counter is necessary here
// because the counter is updated by multiple goroutines.
```

```
type Order struct {
    id uint64
    customer string
    reply chan *Order
    preparedBy string
}
var nextId atomic.Uint64

// The waiter is represented by a channel of orders. The waiter will
// take orders from customers and send them to the cook. The cook will
// then send the prepared meal back to the waiter. To simulate a waiter
// being busy, the waiter channel has a buffer of 3 orders.

var Waiter = make(chan *Order, 3)

// A cook spends their time fetching orders from the order channel,
// cooking the requested meal, and sending the meal back through the
// order's reply channel.

func Cook(name string) {
    log.Println(name, "starting work")
    for order := range Waiter {
        do(10, name, "cooking order", order.id, "for", order.customer)
        order.preparedBy = name
        order.reply <- order
    }
}

// A customer eats five meals and then goes home. Each time they enter the
// restaurant, they place an order with the waiter. If the waiter is too
// busy, the customer will wait for 5 seconds before abandoning the order.
// If the order does get placed, then they will wait as long as necessary
// for the meal to be cooked and delivered.

func Customer(name string, wg *sync.WaitGroup) {
    defer wg.Done()
    ch := make(chan *Order)
    for mealsEaten := 0; mealsEaten < 5; {
        order := &Order{id: nextId.Add(1), customer: name, reply: ch}
        log.Println(name, "placed order", order.id)
        select {
        case Waiter <- order:
            meal := <-ch
            do(2, name, "eating cooked order", meal.id, "prepared by", meal.prepar
            mealsEaten += 1
```

```
        case <-time.After(7 * time.Second):
            do(5, name, "waiting too long, abandoning order", order.id)
        }
    }
    log.Println(name, "going home")
}

func main() {
    var customersDone sync.WaitGroup
    customers := []string{
        "Ani", "Bai", "Cat", "Dao", "Eve", "Fay", "Gus", "Hua", "Iza", "Jai",
    }
    for _, customer := range customers {
        customersDone.Add(1)
        go Customer(customer, &customersDone)
    }
    go Cook("Remy")
    go Cook("Colette")
    go Cook("Linguini")
    customersDone.Wait()
    close(Waiter)
    log.Println("Restaurant closing")
}
```